



PROJETO DE UM SISTEMA EMBARCADO

SMARTPARK

Estacionamento Inteligente com Arduino Uno e Simulação 3D

Integrantes

João Gabriel Barbosa Costa

Gabriel Sales

Antônio Carlos

Professor: Claudio Pereira da Silva

Recife — PE

2026

SMARTPARK — ESTACIONAMENTO INTELIGENTE

Documentação técnica e acadêmica do projeto de um sistema embarcado desenvolvido para a disciplina de Sistemas Embarcados, com foco na simulação de um estacionamento inteligente capaz de detectar a ocupação de vagas, informar a disponibilidade, controlar uma cancela automática e sinalizar situações de lotação.

Trabalho apresentado à Faculdade Nova Roma como parte das atividades da disciplina
de Sistemas Embarcados.

SUMÁRIO

1. APRESENTAÇÃO DO PROJETO	4
2. PROBLEMA E DOR DO CLIENTE	5
3. OBJETIVOS	5
4. VISÃO GERAL DO SISTEMA.....	6
5. TECNOLOGIAS E COMPONENTES	7
6. ARQUITETURA E MODELO DE FUNCIONAMENTO	9
7. FUNCIONAMENTO PASSO A PASSO	10
8. FLUXO DE DECISÃO	11
9. MAPA DE PINOS	11
10. SIMULAÇÃO VIRTUAL NO WOKWI	12
11. LÓGICA DE SOFTWARE	15
12. TESTES E VALIDAÇÃO	19
13. TRATAMENTO DE SITUAÇÕES E LIMITAÇÕES	20
14. EVOLUÇÕES FUTURAS.....	21
15. ORGANIZAÇÃO DA EQUIPE.....	21
16. ORGANIZAÇÃO DO REPOSITÓRIO	22
17. PLANO DE DESENVOLVIMENTO.....	22
18. CRITÉRIOS DE ACEITAÇÃO DO PROJETO	23
19. CONCLUSÃO.....	23
20. REFERÊNCIAS E FONTES.....	24
APÊNDICE — ROTEIRO DE APRESENTAÇÃO.....	24

1. APRESENTAÇÃO DO PROJETO

O SmartPark é um protótipo de sistema embarcado voltado para a automação de um estacionamento de pequeno porte. A proposta nasceu da necessidade de oferecer ao motorista uma informação simples e imediata sobre a disponibilidade de vagas, ao mesmo tempo em que automatiza a entrada por meio de uma cancela.

O projeto foi concebido para atender aos requisitos apresentados na atividade de Projeto de um Sistema Embarcado: utilização de microcontrolador, sensores, atuadores e display; aplicação da criatividade para resolver uma dor de cliente; documentação do código e apresentação de uma solução tecnológica.

O projeto possui dois ambientes independentes: o circuito Arduino Uno no Wokwi, que executa o firmware C++, e a simulação web, que permite controlar carros e cancelas em 3D ou no modo leve. A página não recebe leituras do Wokwi nem de um estacionamento físico.

1.1 Identificação acadêmica

Item	Informação
Instituição	Faculdade Nova Roma
Disciplina	Sistemas Embarcados
Período	5º período
Turma	2026.2
Professor	Claudio Pereira da Silva
Projeto	SmartPark — Estacionamento Inteligente
Equipe	João Gabriel Barbosa Costa; Gabriel Sales; Antônio Carlos
Modalidade inicial	Protótipo e simulação virtual

2. PROBLEMA E DOR DO CLIENTE

Em estacionamentos pequenos ou médios, a ausência de informação em tempo real sobre as vagas pode gerar circulação desnecessária de veículos, formação de fila na entrada e dificuldade para identificar rapidamente onde estacionar. Além disso, uma operação manual da cancela e da contagem de vagas depende da atenção constante de um funcionário.

2.1 Dor principal

- O motorista chega sem saber se existem vagas disponíveis.
- O motorista pode entrar e perder tempo procurando uma vaga.
- A ocupação pode ser controlada manualmente, sujeita a erros.
- A entrada pode continuar liberada mesmo quando o estacionamento está lotado.
- O operador não possui uma visão simples e imediata do estado das vagas.

2.2 Solução proposta

No circuito, quatro sensores HC-SR04 detectam as vagas. O Arduino Uno calcula a disponibilidade, atualiza o LCD e dois LEDs gerais, e controla o servo. A entrada depende de uma solicitação por botão e só é autorizada com vaga disponível e sem falha de sensor. Na página web, o usuário solicita entrada e escolhe uma vaga livre após a abertura da cancela.

3. OBJETIVOS

3.1 Objetivo geral

Desenvolver e documentar um protótipo acadêmico de estacionamento inteligente com Arduino Uno e uma simulação web interativa, demonstrando detecção de vagas, controle de acesso, recusa por lotação e liberação das vagas após a saída.

3.2 Objetivos específicos

- Detectar a presença de veículos nas vagas por meio de sensores ultrassônicos.
- Determinar automaticamente a quantidade de vagas livres.
- Indicar a disponibilidade geral com dois LEDs no circuito e o estado individual das vagas na página web.
- Exibir a disponibilidade geral no display LCD.
- Receber a solicitação de entrada por botão.
- Abrir a cancela após solicitação quando houver vaga e nenhum sensor apresentar falha.
- Impedir a abertura da cancela quando o estacionamento estiver lotado.
- Permitir a escolha da vaga, a saída e uma demonstração automática na simulação web.
- Simular o sistema no ambiente Wokwi.
- Documentar hardware, software, arquitetura, testes e limitações do protótipo.

4. VISÃO GERAL DO SISTEMA

Circuito: sensores das quatro vagas e botões de entrada/saída → Arduino Uno → LCD, dois LEDs e servo.

Página web: comandos do usuário → lógica JavaScript → carros, vagas, cancelas, indicadores e registro de eventos.

Figura 1 — Arquitetura lógica dos dois ambientes independentes.

O Arduino Uno é a unidade central do circuito Wokwi. A página web executa sua própria lógica no navegador, usando HTML, CSS, JavaScript e Three.js. Ela representa o comportamento do estacionamento para demonstração; os sensores da cena são virtuais.

4.1 Fluxo principal

1. Inicializar o Arduino Uno e os periféricos.
2. Ler os sensores das quatro vagas.

3. Classificar cada vaga como LIVRE ou OCUPADA.
4. Contar o total de vagas livres.
5. Atualizar os dois LEDs gerais de disponibilidade.
6. Atualizar o LCD.
7. Verificar os botões de solicitação de entrada e saída.
8. Abrir a cancela na entrada somente com vaga disponível e sem falha.
9. Manter a entrada bloqueada em caso de lotação ou falha e informar o motivo no LCD.
10. Repetir continuamente o ciclo de monitoramento.

5. TECNOLOGIAS E COMPONENTES

5.1 Arduino Uno

O Arduino Uno é a placa utilizada no circuito Wokwi. O firmware lê quatro sensores HC-SR04, recebe os botões de entrada e saída e controla o LCD, dois LEDs gerais e o servo. A versão implementada não utiliza ESP32 nem conectividade Wi-Fi.

5.2 C/C++ com Arduino

A programação utiliza a estrutura típica do ambiente Arduino, com as funções `setup()` e `loop()`. O código foi organizado em funções para facilitar leitura, manutenção e testes.

5.3 Wokwi

O Wokwi simula o Arduino Uno, quatro sensores ultrassônicos, LCD, servo, dois LEDs, resistores e dois botões. A distância de cada sensor pode ser alterada manualmente para representar a ocupação ou liberação da vaga.

5.4 Sensor ultrassônico HC-SR04

Cada HC-SR04 utiliza TRIG para iniciar a medição e ECHO para devolver um pulso proporcional à distância. O firmware calcula centímetros dividindo a duração em microssegundos por 58. Até 18 cm, a vaga é ocupada; a partir de 25 cm, fica livre; entre esses limites, preserva o estado anterior.

5.5 LCD 16x2 com interface I2C

O LCD apresenta informações ao usuário. A interface I2C reduz a quantidade de fios necessários e utiliza SDA e SCL para comunicação.

5.6 Servo motor SG90

O servo representa a cancela. A posição inicial representa a cancela fechada; quando existe vaga, o sistema gira o servo para a posição de abertura e depois retorna à posição inicial.

5.7 LEDs

O circuito possui um LED verde e um vermelho para o estado geral. Verde indica vagas disponíveis e ausência de falhas. Vermelho indica lotação ou falha. A página web mostra também o estado individual das vagas.

5.8 Botões de entrada e saída

Os dois botões usam INPUT_PULLUP. Um novo aperto estável, com debounce de 35 ms, solicita a passagem. A entrada exige disponibilidade e ausência de falhas. A saída pode abrir a cancela mesmo com lotação ou falha. O circuito não possui buzzer.

5.9 Git/GitHub

O repositório <https://github.com/jgbarbosa2356/estacionamento-inteligente> contém o circuito e a simulação web. A página está publicada em <https://jgbarbosa2356.github.io/estacionamento-inteligente/>. O GitHub Pages hospeda os arquivos estáticos HTML, CSS e JavaScript.

Tecnologia/componente	Função	Qtd.
Arduino Uno	Processamento do circuito	1
HC-SR04	Sensor de cada vaga	4
LCD 16x2 I2C	Disponibilidade e mensagens	1
Servo	Cancela do circuito	1
LED verde / vermelho	Disponibilidade geral	2
Resistor 220 Ω	Limitação de corrente dos LEDs	2
Botão	Solicitação de entrada / saída	2
Wokwi	Simulação do circuito	1 ambiente
C++ / Arduino	Firmware	1 código
HTML CSS JavaScript	Interface e lógica web	1 página
Three.js 0.186.1	Cena 3D e animações	1 biblioteca
esbuild 0.25.12	Empacotamento JavaScript	1 ferramenta
GitHub / Pages	Versionamento e publicação	1 repositório

6. ARQUITETURA E MODELO DE FUNCIONAMENTO

As quatro vagas são identificadas de 1 a 4. No circuito, a ocupação vem da distância configurada em cada HC-SR04. Na página 3D, a distância virtual acompanha a posição dos veículos. O modo leve representa a ocupação diretamente por estados livres ou ocupados.

Figura 2 — Organização funcional das quatro vagas.

6.1 Organização das vagas

Cada vaga possui um sensor no circuito. Os dois LEDs indicam o estado geral, enquanto LCD e monitor serial informam disponibilidade e leituras. Na página web, a lista de vagas permite selecionar a posição desejada e consultar seu estado.

Estado	Condição	Indicação	Significado
LIVRE	≥ 25 cm	Lista web; total no LCD	Vaga disponível
OCUPADA	≤ 18 cm	Lista web; monitor serial	Veículo presente
INTERMEDIÁRIO	$18 < \text{cm} < 25$	Estado anterior	Histerese
FALHA	Sem ECHO	LED vermelho e LCD	Entrada bloqueada

6.2 Limite de ocupação

A classificação utiliza histerese: distância ≤ 18 cm significa ocupada; distância ≥ 25 cm significa livre; entre 18 e 25 cm, mantém-se o estado anterior. O circuito agenda uma leitura a cada 450 ms. Os limites deverão ser calibrados antes de uma montagem física.

6.3 Estado global

A variável livres conta apenas sensores válidos cujo estado é livre. falhaGeral indica ausência de retorno em algum sensor. Lotação corresponde a zero vagas disponíveis; uma falha também bloqueia novas entradas, mesmo quando a contagem mostra vagas livres.

7. FUNCIONAMENTO PASSO A PASSO

Inicialização: O Arduino Uno configura os pinos, inicializa o LCD, posiciona a cancela fechada e prepara os sensores.

Leitura das vagas: Cada sensor envia um pulso ultrassônico e o Arduino Uno calcula a distância aproximada.

Classificação: Aplicam-se os limites de 18 e 25 cm; a faixa intermediária conserva o estado anterior. Uma leitura sem retorno marca falha.

Sinalização: O LED verde indica disponibilidade sem falhas. O vermelho indica lotação ou falha.

Contagem: O Arduino Uno soma as vagas livres e atualiza a informação geral.

Display: O LCD mostra LIVRES: n / 4 e mensagens como PRESSIONE BOTAO, CANCELA ABERTA, LOTADO ou FALHA NO SENSOR.

Solicitação de entrada: O motorista pressiona o botão de entrada. Não existe sensor automático de aproximação na versão atual.

Decisão: A entrada só abre com vaga disponível e sem falha. A saída pode abrir independentemente dessas condições.

Abertura: O servo recebe a posição de 90° e o sistema registra o instante com millis().

Fechamento: Após 3 segundos, o servo retorna a 0°. A espera não usa delay(3000), permitindo continuar o ciclo de monitoramento.

Repetição: O sistema retorna à leitura das vagas e mantém o monitoramento contínuo.

8. FLUXO DE DECISÃO

1. Ler V1, V2, V3 e V4.
2. Contar vagas livres.
3. Atualizar LEDs e LCD.
4. Verificar um novo aperto dos botões.
5. Sem solicitação, manter o monitoramento.
6. Na entrada, abrir somente se livres > 0 e falhaGeral for falsa.
7. Com lotação ou falha, recusar a entrada e informar o motivo. Na saída, permitir abertura.
8. Retornar ao início.

9. MAPA DE PINOS

A pinagem abaixo corresponde ao sketch.ino e ao diagram.json do repositório atual.

Função	Pino Uno	Tipo	Observação
--------	----------	------	------------

Função	Pino Uno	Tipo	Observação
Vaga 1 TRIG / ECHO	D2 / D3	Saída / Entrada	HC-SR04
Vaga 2 TRIG / ECHO	D4 / D5	Saída / Entrada	HC-SR04
Vaga 3 TRIG / ECHO	D6 / D7	Saída / Entrada	HC-SR04
Vaga 4 TRIG / ECHO	D8 / D9	Saída / Entrada	HC-SR04
Servo	D10	Saída	0° fechado; 90° aberto
Botão entrada	D11	INPUT_PULLUP	Botão entre pino e GND
Botão saída	D12	INPUT_PULLUP	Botão entre pino e GND
LED verde	A0	Saída	Resistor 220 Ω
LED vermelho	A1	Saída	Resistor 220 Ω
LCD SDA / SCL	A4 / A5	I2C	Endereço 0x27
Alimentação	5V / GND	Alimentação	Conforme diagram.json

Na montagem física, dimensionar a alimentação, usar resistores nos LEDs e manter aterramento comum entre placa, sensores e servo. A simulação valida o comportamento lógico, mas não certifica a instalação elétrica nem a segurança mecânica da cancela.

10. SIMULAÇÃO VIRTUAL NO WOKWI

A simulação virtual permite validar a lógica sem depender inicialmente da montagem física. O Wokwi representa os componentes e permite alterar as leituras dos sensores para simular a aproximação de veículos.

10.1 Montagem virtual

1. Criar um projeto Arduino Uno no Wokwi.
2. Importar o diagram.json do repositório, com quatro HC-SR04.
3. Conferir o LCD 16x2 I2C no endereço 0x27.

4. Conferir o servo no pino D10.
5. Conferir os dois LEDs gerais e os resistores de 220 Ω .
6. Conferir os botões de entrada e saída em D11 e D12.
7. Conferir as conexões pela tabela de pinagem.
8. Copiar sketch.ino e libraries.txt para os arquivos correspondentes.
9. Iniciar a simulação e alterar as distâncias dos sensores conforme o cenário.

10.2 Cenário 1 — estacionamento vazio

Configure os quatro sensores para 100 cm. O LCD deve mostrar 4 vagas livres, o LED verde deve acender e a cancela deve permanecer fechada até uma solicitação.

10.3 Cenário 2 — uma vaga ocupada

Altere o sensor da vaga 1 para 10 cm. A contagem deve cair para 3. O LED verde geral continua aceso porque ainda existem vagas e não há falha.

10.4 Cenário 3 — estacionamento lotado

Configure os quatro sensores para 10 cm. A contagem chega a zero e o LED vermelho acende. Pressione ENTRADA: a cancela deve permanecer fechada e o LCD deve informar lotação.

10.5 Cenário 4 — entrada autorizada

Com ao menos uma vaga livre e sensores válidos, pressione ENTRADA. O servo vai a 90° e retorna a 0° após 3 segundos. Para representar o veículo estacionado, altere depois a distância da vaga escolhida para 10 cm. O LCD não contém menu de seleção de vaga.

10.6 Simulação web 3D e modo leve

Acesse <https://jgbarbosa2356.github.io/estacionamento-inteligente/>. Solicite entrada; com disponibilidade, a cancela abre e você escolhe uma vaga livre na lista ou na cena.

O carro percorre a pista e estaciona. Quando as quatro vagas estão ocupadas, uma nova entrada é recusada. Para liberar uma vaga, solicite saída e escolha o carro estacionado. A página permite mudar a câmera, usar vista superior e ajustar o zoom.

A cena utiliza Three.js e renderização WebGL. O modo leve é acessado em ?modo=leve e representa a mesma sequência de entrada, escolha de vaga, lotação e saída em uma interface 2D. No 3D, a ocupação é derivada da posição do carro e de sensores virtuais; no modo leve, a seleção altera diretamente o estado da vaga.

Nenhum desses ambientes envia comandos ao Arduino.

10.7 Demonstração automática completa

O botão Iniciar demonstração completa reinicia a simulação e executa dez etapas: 1) estacionamento vazio; 2 a 5) quatro carros solicitam entrada e ocupam as vagas; 6) uma quinta entrada é recusada por lotação; 7 a 10) os quatro carros saem, liberando todas as vagas. O progresso e os eventos são mostrados na tela. A demonstração também pode ser aberta pelo endereço ?demo=1.

Pausar interrompe o avanço da demonstração; Continuar retoma o fluxo. Reiniciar volta ao estado inicial e ao modo manual. Durante a demonstração, a seleção manual de vagas fica desabilitada para evitar conflitos. Ao concluir, o estacionamento volta a ficar vazio e pode ser testado manualmente ou demonstrado novamente.

10.8 Uso no celular

A interface se reorganiza em telas pequenas, oferece botões de toque e seleção pela lista de vagas. O navegador precisa suportar a renderização necessária à cena 3D. Se houver lentidão ou incompatibilidade, use Modo leve. Não é necessário instalar um aplicativo; a execução completa em celular real ainda precisa ser validada pela equipe.

11. LÓGICA DE SOFTWARE

11.1 Funções principais

Função	Responsabilidade
foiPressionado()	Detecta novo aperto com debounce de 35 ms.
medirDistancia()	Lê HC-SR04; timeout de 30 ms.
atualizarSensores()	Classifica vagas, falhas, contagem e LEDs.
mostrarMensagem()	Define mensagem temporária.
abrirCancela()	Move servo e registra horário de abertura.
atualizarDisplay()	Atualiza linhas do LCD.
setup()	Inicializa circuito e lê sensores.
loop()	Agenda leituras, trata botões e fecha cancela com millis().

11.2 Código-fonte do protótipo

O código abaixo corresponde ao sketch.ino atual do repositório. As bibliotecas utilizadas são Wire, LiquidCrystal_I2C e Servo. A seleção gráfica de vagas e a demonstração automática pertencem ao JavaScript da página web, não a este firmware.

```

/*
  Estacionamento Inteligente – projeto de sistema embarcado
  Equipe: Joao Gabriel Barbosa Costa, Gabriel Sales, Antonio Carlos

  Quatro sensores HC-SR04 monitoram as vagas. A cancela de entrada
  abre apenas quando existe vaga e todos os sensores respondem.
  O botao de saida abre a cancela independentemente da lotacao.
  A posicao real dos carros e determinada pelos sensores: apertar
  um botao nao altera artificialmente a contagem de vagas.
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

const byte NUM_VAGAS = 4;

```

```

const byte TRIG[NUM_VAGAS] = {2, 4, 6, 8};
const byte ECHO[NUM_VAGAS] = {3, 5, 7, 9};
const byte PIN_SERVO = 10;
const byte BOTAO_ENTRADA = 11;
const byte BOTAO_SAIDA = 12;
const byte LED_VERDE = A0;
const byte LED_VERMELHO = A1;

const float LIMIAR_OCUPADA_CM = 18.0;
const float LIMIAR_LIVRE_CM = 25.0; // histerese evita oscilacao
const unsigned long INTERVALO_LEITURA_MS = 450;
const unsigned long TEMPO_CANCELA_MS = 3000;
const unsigned long DEBOUNCE_MS = 35;

LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo cancela;
bool ocupada[NUM_VAGAS] = {false, false, false, false};
bool falhaSensor[NUM_VAGAS] = {false, false, false, false};
byte livres = NUM_VAGAS;
bool falhaGeral = false;
bool aberta = false;
unsigned long aberturaEm = 0;
unsigned long ultimaLeitura = 0;
unsigned long mensagemAte = 0;
const char* mensagem = "";

struct Botao {
    byte pin;
    bool ultimaLeitura;
    bool estadoEstavel;
    unsigned long mudouEm;
};

Botao entrada = {BOTAO_ENTRADA, HIGH, HIGH, 0};
Botao saida = {BOTAO_SAIDA, HIGH, HIGH, 0};

// Retorna true somente no instante de um novo aperto estavel.
bool foiPressionado(Botao &b, unsigned long agora) {
    bool leitura = digitalRead(b.pin);
    if (leitura != b.ultimaLeitura) b.mudouEm = agora;
    b.ultimaLeitura = leitura;
    if (agora - b.mudouEm >= DEBOUNCE_MS && leitura != b.estadoEstavel) {
        b.estadoEstavel = leitura;
        return leitura == LOW;
    }
    return false;
}

// Zero indica ausencia de ECHO: estado desconhecido e entrada bloqueada.
float medirDistancia(byte i) {
    digitalWrite(TRIG[i], LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG[i], HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG[i], LOW);
    unsigned long duracao = pulseIn(ECHO[i], HIGH, 30000UL);
    if (duracao == 0) return 0;
    return duracao / 58.0;
}

```

```

}

void atualizarSensores() {
    livres = 0;
    falhaGeral = false;
    for (byte i = 0; i < NUM_VAGAS; i++) {
        float cm = medirDistancia(i);
        falhaSensor[i] = cm == 0;
        if (falhaSensor[i]) {
            falhaGeral = true;
        } else {
            if (cm <= LIMIAR_OCUPADA_CM) ocupada[i] = true;
            else if (cm >= LIMIAR_LIVRE_CM) ocupada[i] = false;
            if (!ocupada[i]) livres++;
        }
        Serial.print("Vaga "); Serial.print(i + 1);
        Serial.print(": ");
        if (falhaSensor[i]) Serial.println("FALHA SENSOR");
        else {
            Serial.print(cm, 1); Serial.print(" cm - ");
            Serial.println(ocupada[i] ? "OCUPADA" : "LIVRE");
        }
    }
    Serial.print("Livres: "); Serial.println(livres);
    digitalWrite(LED_VERDE, !falhaGeral && livres > 0);
    digitalWrite(LED_VERMELHO, falhaGeral || livres == 0);
}

void mostrarMensagem(const char* texto, unsigned long agora) {
    mensagem = texto;
    mensagemAte = agora + 2200;
}

void abrirCancela(const char* motivo, unsigned long agora) {
    cancela.write(90);
    aberta = true;
    aberturaEm = agora;
    mostrarMensagem(motivo, agora);
    Serial.println(motivo);
}

void atualizarDisplay(unsigned long agora) {
    lcd.setCursor(0, 0);
    lcd.print("LIVRES: "); lcd.print(livres);
    lcd.print(" / 4 ");
    lcd.setCursor(0, 1);
    const char* linha = "PRESSIONE BOTAO";
    if (aberta) linha = "CANCELA ABERTA";
    else if (agora < mensagemAte) linha = mensagem;
    else if (falhaGeral) linha = "FALHA NO SENSOR";
    else if (livres == 0) linha = "LOTADO";
    lcd.print(linha);
    byte len = strlen(linha);
    while (len++ < 16) lcd.print(' ');
}

void setup() {
    Serial.begin(115200);

```

```

for (byte i = 0; i < NUM_VAGAS; i++) {
    pinMode(TRIG[i], OUTPUT);
    pinMode(ECHO[i], INPUT);
}
pinMode(BOTAO_ENTRADA, INPUT_PULLUP);
pinMode(BOTAO_SAIDA, INPUT_PULLUP);
pinMode(LED_VERDE, OUTPUT);
pinMode(LED_VERMELHO, OUTPUT);
cancela.attach(PIN_SERVO);
cancela.write(0);
lcd.init();
lcd.backlight();
atualizarSensores();
atualizarDisplay(millis());
}

void loop() {
    unsigned long agora = millis();
    if (agora - ultimaLeitura >= INTERVALO_LEITURA_MS) {
        ultimaLeitura = agora;
        atualizarSensores();
    }
    if (foiPressionado(entrada, agora)) {
        if (aberta) mostrarMensagem("AGUARDE...", agora);
        else if (falhaGeral) mostrarMensagem("SENSOR COM FALHA", agora);
        else if (livres == 0) mostrarMensagem("ESTAC. LOTADO", agora);
        else abrirCancela("ENTRADA LIBERADA", agora);
    }
    if (foiPressionado(saida, agora)) {
        if (!aberta) abrirCancela("SAIDA LIBERADA", agora);
    }
    if (aberta && agora - aberturaEm >= TEMPO_CANCELA_MS) {
        cancela.write(0);
        aberta = false;
        Serial.println("Cancela fechada");
    }
    atualizarDisplay(agora);
}

```

12. TESTES E VALIDAÇÃO

A validação deve separar revisão de código, execução da simulação e montagem física. Na execução anterior do projeto, o circuito compilou e iniciou com quatro vagas livres; o modo leve completou entrada, lotação, saída, demonstração automática, pausa e reinício. Não há registro de teste físico nem de validação completa da cena 3D em um celular real. A tabela seguinte apresenta casos que devem ser reproduzidos e registrados na apresentação.

ID	Cenário	Entrada	Resultado esperado
----	---------	---------	--------------------

ID	Cenário	Entrada	Resultado esperado
T01	Vazio	4 sensores em 100 cm	4 livres; LED verde
T02	Vaga 1 ocupada	V1 em 10 cm	3 livres
T03	Histerese	Ocupada em 10; depois 22 cm	Mantém ocupada
T04	Lotado	4 sensores em 10 cm	0 livres; LED vermelho
T05	Entrada disponível	Pressionar entrada	90° e retorno em 3 s
T06	Entrada lotada	Pressionar entrada com 0 livres	Cancela fechada; mensagem
T07	Falha de sensor	Ausência de ECHO	Novas entradas bloqueadas
T08	Saída com lotação	Pressionar saída	Cancela abre
T09	Web manual	Solicitar entrada e escolher vaga	Carro ocupa vaga selecionada
T10	Web automático	Iniciar demonstração completa	Enche, recusa quinto e esvazia
T11	Pausa e reinício	Parar; continuar; reiniciar	Pausa, retoma e restaura início
T12	Celular / 3D	Abrir em dispositivo real	Verificar toque, câmera e desempenho

13. TRATAMENTO DE SITUAÇÕES E LIMITAÇÕES

13.1 Sensor sem retorno

A medição possui timeout de 30 ms. Se não houver ECHO, `medirDistancia()` retorna zero, o sensor fica com falha, não é contado como vaga disponível e `falhaGeral` bloqueia novas entradas. A solicitação de saída continua permitida. A ausência de eco não comprova que a vaga esteja livre.

13.2 Calibração

Os limites de 18 e 25 cm são parâmetros do protótipo. A histerese reduz alternâncias perto de um único limite. A instalação física exige calibração considerando posição do sensor e características dos veículos.

13.3 Alimentação

Na montagem física, a alimentação dos sensores e do servo deve ser dimensionada corretamente. O servo pode gerar variações de corrente e ruído, portanto a equipe deve considerar fonte adequada e aterramento comum.

13.4 Limites do protótipo

- O projeto não implementa pagamento.
- Não existe cadastro de veículos.
- Não há reconhecimento de placa.
- Não há banco de dados.
- A página pode ser acessada pelo navegador do celular; não existe aplicativo nativo.
- A simulação não substitui a validação elétrica do circuito físico.
- O sistema foi dimensionado como protótipo acadêmico de quatro vagas.

14. EVOLUÇÕES FUTURAS

- Integrar um controlador físico a um serviço que publique leituras reais, eventualmente migrando para uma placa com rede, como ESP32.
- Conectar o painel web aos sensores físicos; atualmente a página mantém estados virtuais independentes.
- Adicionar identificação por RFID.
- Adicionar sensor específico para detectar entrada e saída.
- Criar histórico de ocupação.
- Adicionar contador de tempo de permanência.
- Implementar cadastro de veículos autorizados.

- Adicionar reconhecimento de placas em uma versão avançada.
- Adicionar energia solar para um protótipo autônomo.
- Expandir de quatro para várias vagas.
- Criar aplicativo para consultar disponibilidade.

15. ORGANIZAÇÃO DA EQUIPE

Integrante	Responsabilidades sugeridas
João Gabriel Barbosa Costa	Integração, programação do Arduino e documentação técnica (divisão sugerida).
Gabriel Sales	Montagem/simulação dos componentes, sensores, LEDs e testes.
Antônio Carlos	Documentação, arquitetura, registro dos testes e preparação da apresentação.

A divisão é uma proposta de organização e não impede que os integrantes atuem em conjunto nas etapas de desenvolvimento, testes e apresentação.

16. ORGANIZAÇÃO DO REPOSITÓRIO

```
estacionamento-inteligente/
  README.md
  sketch.ino
  diagram.json
  libraries.txt
  index.html
  simulacao_visual.html
  parking3d-source.js
  parking3d.js
  package.json
  THREE-LICENSE.txt
```

Os arquivos abaixo correspondem à organização atual. README.md contém instruções de execução, componentes e roteiro. O circuito e a página são mantidos no mesmo repositório, mas executam separadamente.

17. PLANO DE DESENVOLVIMENTO

Etapa	Atividade	Resultado
1	Definição do problema	Dor do cliente documentada
2	Arquitetura	Diagrama do sistema
3	Montagem virtual	Circuito no Wokwi
4	Leitura dos sensores	Estados das vagas
5	LCD e LEDs	Interface visual
6	Servo e entrada	Cancela automática
7	Integração	Sistema completo
8	Testes	Evidências de funcionamento
9	Documentação	Relatório e README
10	Apresentação	Demonstração do protótipo

18. CRITÉRIOS DE ACEITAÇÃO DO PROJETO

- O Arduino Uno deve iniciar sem erro.
- As quatro vagas devem ser identificadas individualmente.
- A página web deve identificar e mostrar a vaga ocupada.
- A página web deve permitir selecionar uma vaga livre após a solicitação de entrada.
- A contagem de vagas deve refletir os estados detectados.
- O LCD deve apresentar a quantidade de vagas livres.
- Quando todas as vagas estiverem ocupadas, o LCD deve indicar lotação.
- Com vagas e sensores válidos, uma solicitação de entrada deve abrir a cancela.
- Com o estacionamento lotado, a cancela deve permanecer fechada.
- O sistema deve retornar ao monitoramento após cada evento.

- A documentação deve permitir que outro integrante reproduza a simulação.

19. CONCLUSÃO

O SmartPark apresenta uma aplicação prática de sistemas embarcados para um problema cotidiano: a identificação e o controle da disponibilidade de vagas em um estacionamento. A solução utiliza sensores para obter informações do ambiente, um microcontrolador para processar essas informações, um display para comunicação com o usuário e atuadores para automatizar a entrada.

O circuito Wokwi permite estudar a aquisição de distâncias e o controle dos periféricos. A simulação web facilita a apresentação do fluxo ao motorista, com escolha de vaga, saída e demonstração automática. A integração entre ambos é uma evolução futura.

Dessa forma, o protótipo atende ao conceito central da atividade: utilizar um sistema microcontrolado, sensores, atuadores e display para criar uma solução tecnológica orientada a um problema real.

20. REFERÊNCIAS E FONTES

ARDUINO. UNO R: <https://docs.arduino.cc/hardware/uno-rev3/>.

WOKWI. HC-SR04 Reference: <https://docs.wokwi.com/parts/wokwi-hc-sr04>.

THREE.JS. Documentation: <https://threejs.org/docs/>.

GITHUB. What is GitHub Pages: <https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages>.

EQUIPE DO PROJETO. Estacionamento inteligente. Código-fonte:

<https://github.com/jgbarbosa2356/estacionamento-inteligente>.

APÊNDICE — ROTEIRO DE APRESENTAÇÃO

1. Apresentar o problema: motorista não sabe se há vaga.
2. Mostrar a arquitetura do SmartPark.
3. Abrir a página publicada e mostrar o circuito Wokwi como ambiente independente.
4. Mostrar quatro vagas livres.
5. No Wokwi, ajustar o sensor da vaga 1 para 10 cm e mostrar a redução da contagem.
6. Repetir até deixar todas as vagas ocupadas.
7. Mostrar a mensagem de estacionamento lotado.
8. Pressionar ENTRADA no circuito e demonstrar a recusa por lotação.
9. Na página, iniciar a demonstração completa: quatro entradas, quinta entrada recusada e quatro saídas.
10. Usar Pausar/Continuar para explicar as etapas e Reiniciar para voltar ao início.

11. Concluir mostrando a lógica do Arduino, os sensores e as funcionalidades web.